Deepfake Image Detection System with Blockchain and Machine Learning

AI23331 - FUNDAMENTALS OF MACHINE LEARNING MINIPROJECT REPORT

Submitted by

C PRITHIVIRAAJ **(2116230701508)**

MUSA HAJI **(2116230701391)**

SHANMUGANATHAN S **(2116230701307)**

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

RAJALAKSHMI ENGINEERING COLLEGE ANNA UNIVERSITY,

CHENNAI MAY 2026

RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

BONAFIDE CERTIFICATE

Certified that this Project titld **Deepfake Image Detection System with Blockchain and Machine Learning"** is the bonafide work of **"C PRITHIVIRAAJ (2116230701508), MUSA HAJI(2116230701391), SHANMUGANATHAN S (2116230701307)"** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. J. Jinu Sophia., M.E., Ph.D.,

Associate Professor

Department of Computer Science and Engineering,

Rajalakshmi Engineering College, Chennai-602 105.

Submitted to Mini Project Viva-Voce Examination held on _____

Internal Examiner **External Examiner**

ABSTRACT

Deepfake technology has rapidly evolved with the advancement of Artificial Intelligence and Deep Learning techniques, enabling the creation of highly realistic fake images and videos. Although this technology has useful applications in entertainment and media industries, it also poses serious threats such as misinformation, identity theft, cybercrime, and digital manipulation. Detecting deepfake content has therefore become an important research area in the field of Machine Learning and Cybersecurity.

The proposed project, titled **"Deepfake Image Detection System with Blockchain and Cloud Integration"**, aims to identify whether an uploaded image is real or fake using Deep Learning algorithms. The system utilizes Convolutional Neural Networks (CNN) developed using TensorFlow and Keras for image classification. The model is trained using deepfake datasets containing both manipulated and original images. Image preprocessing techniques such as resizing, normalization, and augmentation are performed to improve prediction accuracy and model performance.

A Flask-based web application is developed to provide an interactive user interface where users can upload images for verification. Once an image is uploaded, the trained model predicts whether the image is REAL or FAKE and displays the confidence score. To ensure data integrity and security, SHA-256 hashing and Blockchain mechanisms are integrated into the system. Every prediction result is stored as a block containing image details, prediction information, confidence values, and file hash. This improves transparency and prevents tampering of records.

In addition, cloud integration is implemented using Amazon Web Services (AWS) S3 storage for secure image uploading and storage. The cloud component enables scalability, remote accessibility, and efficient management of uploaded images. The system also includes proper error handling, image orientation correction, and secure file management for reliable operation.

The proposed solution combines Artificial Intelligence, Blockchain, and Cloud Computing technologies into a unified framework for secure and efficient deepfake detection. This project demonstrates the practical application of machine learning techniques in solving real-world cybersecurity problems while ensuring data authenticity and cloud-based accessibility.

ACKNOWLEDGMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S. MEGANATHAN, B.E, F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.,** and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN**, **Ph.D.,** for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.,** our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. E. M. MALATHY, M.E., Ph.D.**, Professor and Head of the Department of Computer Science and Engineering for his guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide **Dr. J. JINU SOPHIA, M.E., Ph.D.,** Associate Professor Department of Computer Science and Engineering for her useful tips during our review to build our project.

C PRITHIVIRAAJ 2116230701508

MUSA HAJI 2116230701391

SHANMUGANATHAN S 2116230701307

Contents

List of Figures

List of Tables

ix

List of Abbreviations

Abbreviation		Expansion
AI	–	Artificial Intelligence
API	–	Application Programming Interface
AUC	–	Area Under the Curve
AWS	–	Amazon Web Services
CNN	–	Convolutional Neural Network
CPU	–	Central Processing Unit
CUDA	–	Compute Unified Device Architecture
DCGAN	–	Deep Convolutional Generative Adversarial Network
DFD	–	Data Flow Diagram
DFDC	–	DeepFake Detection Challenge
DL	–	Deep Learning
DNN	–	Deep Neural Network
EC2	–	Elastic Compute Cloud
ELB	–	Elastic Load Balancer
ER	–	Entity Relationship

FN	–	False Negative
FP	–	False Positive
FPN	–	Feature Pyramid Network
GAN	–	Generative Adversarial Network
GPU	–	Graphics Processing Unit
HTML	–	HyperText Markup Language
HTTP	–	HyperText Transfer Protocol
IAM	–	Identity and Access Management
JSON	–	JavaScript Object Notation
JWT	–	JSON Web Token
ML	–	Machine Learning
MLP	–	Multi-Layer Perceptron
MTCNN	–	Multi-task Cascaded Convolutional Network
REST	–	Representational State Transfer
ReLU	–	Rectified Linear Unit
ROC	–	Receiver Operating Characteristic

LIST OF ABBREVIATIONS

Abbreviation		Expansion
S3	–	Simple Storage Service
SDK	–	Software Development Kit
SRS	–	Software Requirements Specification
SSL	–	Secure Sockets Layer
TN	–	True Negative
TP	–	True Positive
UI	–	User Interface
URL	–	Uniform Resource Locator
VGG	–	Visual Geometry Group
VPC	–	Virtual Private Cloud
WSGI	–	Web Server Gateway Interface
XAI	–	Explainable Artificial Intelligence

CHAPTER 1

Introduction

1.1 Background and Motivation

The digital revolution of the twenty-first century has fundamentally altered the way information is created, disseminated, and consumed. While this transformation has enabled unprecedented access to knowledge and communication, it has simultaneously introduced vulnerabilities that malicious actors are increasingly exploiting. Among the most alarming developments in recent years is the emergence of *deepfake* technology — a class of synthetic media in which deep learning algorithms are used to create hyper-realistic images and videos that convincingly depict events, statements, or appearances that never occurred.

The term *deepfake* originates from a combination of "deep learning" and "fake," and broadly refers to any AI-generated or AI-manipulated digital content that misrepresents reality. From face-swapping in political speeches to fabricating evidence in judicial proceedings, the misuse potential of this technology is both broad and deeply troubling. A 2023 report by *Sensity AI* estimated that the total number of deepfake videos circulating online has been doubling every six to twelve months since 2018, with over 500,000 such videos identified by late 2023.

The problem is further compounded by the democratization of deepfake creation tools. Platforms such as *DeepFaceLab*, *Reface*, and *FaceSwap* have brought professional-grade synthesis capabilities to non-expert users, requiring little more than a consumer-grade GPU and a modest dataset of target images. As a result, the barrier to creating convincing synthetic media has dropped dramatically, increasing both the volume and sophistication of deepfake content.

1.2 Deepfake Technology

Deepfake generation relies on a family of deep learning architectures, the most prominent of which is the Generative Adversarial Network (GAN), introduced by Goodfellow et al. in 2014 [1]. A GAN consists of two neural networks trained in opposition: a *generator* that synthesizes increasingly realistic images, and a *discriminator* that attempts to distinguish synthetic from genuine samples. Over successive training iterations, the generator learns to produce outputs that the discriminator cannot reliably classify as fake, resulting in photorealistic synthetic images.

2

1.2.1 Face-Swapping Techniques

The most common form of deepfake involves *face-swapping*, where the facial region of a target video is replaced with a synthesized face derived from a source individual. This process typically involves three stages:

1. Face Extraction and Alignment: Frames are extracted from source and target videos, facial landmarks are detected, and faces are aligned geometrically.
2. Face Synthesis: An encoder-decoder architecture learns a shared latent representation of both faces and generates the swapped face by combining the decoded appearance of the source with the pose and expression of the target.
3. Blending and Post-Processing: The synthetic face is blended back into the target frame using techniques such as Poisson blending, colour correction, and sharpening to minimize visible artifacts.

1.2.2 Other Synthesis Techniques

Beyond face-swapping, deepfake generation encompasses:

- Face Reenactment: Modifying facial expressions and head movements to make a target appear to speak or emote differently (e.g., *Face2Face* [22]).
- Attribute Manipulation: Altering attributes such as age, gender, or hair colour using conditional GANs (e.g., *StarGAN* [23]).
- Full Body Synthesis: Generating entire human figures in novel poses or environments using diffusion models (e.g., *DALL-E*, *Stable Diffusion*).

1.3 Artificial Intelligence and Machine Learning in Image Analysis

Artificial intelligence, and more specifically machine learning, has become the cornerstone of modern computer vision. Convolutional Neural Networks (CNNs), introduced by LeCun et al. in 1998 [20] and popularised by the AlexNet victory in ILSVRC 2012 [21], have proven to be extraordinarily effective at extracting hierarchical spatial features from images. Modern CNN architectures such as VGG-16 [19], ResNet [18], and EfficientNet [9] achieve nearhuman performance on standard visual recognition benchmarks.

Transfer learning — the practice of fine-tuning a model pre-trained on a large dataset (e.g.,

ImageNet) on a smaller domain-specific dataset — has been particularly transformative. It allows researchers to leverage learned feature representations for new tasks without requiring prohibitively large training datasets or compute resources.

In the context of deepfake detection, AI models must learn to identify subtle visual artifacts that differentiate authentic from synthesized images. These include:

- Irregular skin textures and lighting inconsistencies
- Blending artifacts at facial boundaries
- Temporal inconsistencies in video (flickering, unnatural blinks)
- Frequency-domain anomalies introduced by GAN upsampling
- Physiological signals absent in synthetic faces (e.g., blood-flow-based colour variation)

1.4 Image Forgery and Digital Manipulation

Image forgery predates deep learning and encompasses classical manipulation techniques such as *copy-move forgery* (duplicating regions within an image), *splicing* (combining regions from multiple images), and *inpainting* (filling in or removing regions). Digital forensics has developed a range of tools for detecting these manipulations by examining metadata, compression artifacts, noise patterns, and statistical properties of image pixels.

The advent of AI-based synthesis has rendered many traditional forensic approaches insufficient. GAN-generated images exhibit different statistical properties from classically manipulated images, and they often lack the detectable noise inconsistencies that traditional forensic tools rely upon. This has necessitated the development of new, learning-based detection paradigms.

1.5 Cloud Computing Overview

Cloud computing refers to the on-demand delivery of computing resources — including servers, storage, databases, networking, software, and analytics — over the internet, typically on a pay-as-you-go basis. It eliminates the need for organizations to maintain costly on-premises infrastructure and enables rapid scaling of resources in response to workload demands.

The National Institute of Standards and Technology (NIST) defines cloud computing through five essential characteristics: on-demand self-service, broad network access, resource pooling, rapid elasticity, and measured service.

1.5.1 Service Models

- Infrastructure as a Service (IaaS): Provides virtualized computing resources (VMs, storage, networks). Example: AWS EC2, Azure VMs.
- Platform as a Service (PaaS): Provides development and deployment environments. Example: AWS Elastic Beanstalk, Google App Engine.
- Software as a Service (SaaS): Delivers complete applications over the internet. Example: Google Workspace, Salesforce.

1.5.2 Deployment Models

- Public Cloud: Resources owned and operated by a third-party provider. Example: AWS, Azure, GCP.
- Private Cloud: Resources dedicated to a single organization, either on-premises or hosted.
- Hybrid Cloud: Combination of public and private clouds.

1.6 Amazon Web Services (AWS)

Amazon Web Services is the world's most comprehensive and broadly adopted cloud platform, offering over 200 fully featured services from data centres globally. AWS holds approximately 32% of the global cloud infrastructure market share as of 2024, making it the dominant provider.

For this project, the following AWS services are utilized:

Table 1.1: AWS Services Used in the Project

AWS Service	Purpose in This Project
EC2	Hosting the Flask web application and serving ML model inference
S3	Storing uploaded images, processed results, and model artefacts
IAM	Managing user roles, permissions, and secure API access
CloudFront	Content Delivery Network for low-latency frontend delivery
CloudWatch	Monitoring application metrics, logs, and perfor-

	mance alerts
VPC	Isolated network environment for secure deployment
Elastic IP	Static public IP address for the EC2 instance
Security Groups	Firewall rules controlling inbound/outbound traffic

1.7 Problem Statement

The growing volume and realism of AI-generated deepfake imagery have outpaced the development of reliable, accessible detection tools. Existing academic detection models are often:

- Trained on limited, homogeneous datasets that do not generalize to novel deepfake methods
- Deployed only as offline scripts without practical user interfaces
- Not scalable to handle real-world production workloads
- Lacking integration with cloud infrastructure for reliable and distributed access

There is a pressing need for a system that combines high-accuracy deep learning detection with a production-grade cloud deployment, making deepfake detection accessible to media professionals, researchers, and end users at scale.

1.8 Existing System

Existing deepfake detection systems fall into two broad categories. The first comprises researchoriented models, such as MesoNet [2], FaceXray [5], and Xception-based classifiers [3], which achieve high accuracy in controlled laboratory settings but are rarely made available as deployable applications. The second category includes commercial services (e.g., Sensity AI, Microsoft Video Authenticator) that are proprietary, expensive, and not accessible to independent researchers or smaller organizations.

Limitations of existing systems:

- No integrated cloud deployment or REST API
- Limited explainability of detection decisions
- Poor generalization across different deepfake generation techniques

- Absence of real-time processing capability for high-volume uploads
- Restricted access due to proprietary licensing

1.9 Proposed System

The proposed Deepfake Image Detection System with Cloud Computing Integration addresses the above limitations through the following design principles:

1. A CNN-based detection model using EfficientNet-B4 as the backbone, fine-tuned on a large-scale, diverse deepfake dataset.
2. A Flask REST API that encapsulates the model, providing a stateless, scalable inference endpoint.
3. A responsive web interface allowing users to upload images and receive classification results with confidence scores and visual explanations.
4. AWS cloud deployment using EC2 for compute, S3 for storage, IAM for security, and CloudWatch for observability.
5. Grad-CAM visualization to highlight image regions that contributed most to the detection decision, enhancing interpretability.

1.10 Objectives

The primary objectives of this project are:

1. To study and analyze the state-of-the-art techniques in deepfake generation and detection.
2. To design and implement a high-accuracy CNN-based deepfake image classifier.
3. To develop a production-ready Flask-based web application that wraps the detection model.
4. To deploy the complete system on AWS cloud infrastructure with appropriate security configurations.
5. To evaluate the system's performance using standard metrics including accuracy, precision, recall, and F1-score.
6. To generate visual explanations of model decisions using Grad-CAM.

1.11 Scope of the Project

The scope encompasses the full lifecycle of a machine learning application: data acquisition, preprocessing, model development, training, evaluation, API development, frontend design, and cloud deployment. The system operates on static images (JPEG/PNG format) of human faces. Video processing, while proposed as a future enhancement, is outside the current scope.

The cloud deployment is designed for the AWS platform, specifically within the ap-south-1 (Mumbai) region for compliance and low latency within India.

1.12 Applications

- Journalism and Media Verification: News organizations can verify the authenticity of images before publication.
- Social Media Moderation: Platforms can automatically flag and remove deepfake content.
- Digital Forensics: Law enforcement agencies can use the system as a forensic tool in legal proceedings.
- Identity Verification: Financial institutions and KYC providers can validate submitted identity documents and selfies.
- Academic Research: Researchers can use the API to benchmark new detection or generation methods.
- Personal Safety: Individuals can verify whether images purportedly showing them have been manipulated.

1.13 Advantages of the Proposed System

- High classification accuracy (97.4%) across multiple deepfake generation techniques
- Scalable cloud deployment supporting concurrent user requests
- Interpretable results via Grad-CAM activation maps
- Zero setup required for end users (web-based interface)
- Cost-effective pay-as-you-go cloud billing model
- Secure data handling with AWS IAM and S3 encryption

1.14 Challenges

- Dataset Diversity: Ensuring the training dataset covers all major deepfake generation methods to prevent overfitting to specific GAN architectures.
- Class Imbalance: Real images typically outnumber fake ones in naturally collected datasets, requiring careful balancing strategies.
- Evolving Threat Landscape: Deepfake generation methods improve continuously; detection models require periodic retraining.
- Adversarial Attacks: Deepfakes can be specifically crafted to evade detection systems.
- Cloud Cost Management: GPU-based inference on EC2 can be expensive at scale; model optimization (quantization, pruning) is necessary.
- Privacy Concerns: Processing and storing facial images requires strict compliance with data protection regulations.

CHAPTER 2

Literature Survey

2.1 Introduction

A comprehensive literature survey was conducted to understand the state of the art in deepfake generation, deepfake detection, and cloud-based deployment of deep learning systems. This chapter reviews fifteen significant research contributions, analyzing their methodologies, datasets, performance metrics, and limitations. The survey informs the design decisions adopted in the proposed system and identifies the research gaps this work seeks to address.

2.2 Review of Related Works 2.2.1 MesoNet: A Compact Facial Video Forgery Detection Network (Afchar et al., 2018)

Afchar et al. [2] proposed MesoNet, a lightweight CNN specifically designed for deepfake detection at the mesoscopic level — intermediate spatial scales between pixel-level noise analysis and high-level semantic features. The network consists of two variants, Meso-4 and MesoInception-4, both comprising fewer than 30,000 parameters. Trained on a private dataset of Deepfake and Face2Face manipulations, MesoNet achieved accuracies of 98% (Deepfake) and 95% (Face2Face).

Advantages: Extremely lightweight, fast inference, suitable for real-time applications. Limitations: Poor generalization to novel manipulation types not seen during training; no cloud deployment; limited dataset.

2.2.2 FaceForensics++: Learning to Detect Manipulated Facial Images (Rössler et al., 2019)

Rössler et al. [3] introduced the FaceForensics++ dataset, a landmark contribution comprising 1,000 original YouTube videos manipulated using four methods: Deepfakes, Face2Face, FaceSwap, and NeuralTextures. They benchmarked several detection approaches and demonstrated that an Xception-based [17] classifier achieves over 99% accuracy on high-quality manipulations, though performance degrades significantly under compression.

Advantages: Large, standardized, publicly available dataset; rigorous benchmarking. Limitations: Detection accuracy degrades under social-media-level video compression; no deployment framework.

2.2.3 Celeb-DF: A Large-Scale Challenging Dataset for DeepFake Forensics (Li et al., 2020)

Li et al. [4] addressed quality limitations in early deepfake datasets by curating Celeb-DF, comprising 590 original celebrity videos and 5,639 corresponding deepfake videos generated using an improved synthesis pipeline. Detection models trained on FaceForensics++ achieved only 53–73% AUC on Celeb-DF, highlighting the cross-dataset generalization problem.

Advantages: High visual quality deepfakes; significant challenge for existing detectors.

Limitations: Restricted to celebrity faces; limited diversity in subjects.

2.2.4 Face X-Ray for More General Face Forgery Detection (Li & Lyu, 2020)

Li and Lyu [5] proposed Face X-ray, which detects blending boundaries in manipulated images regardless of the specific manipulation method used. By training on intrinsic blending artifacts rather than GAN-specific features, Face X-ray generalizes significantly better to unseen deepfake techniques.

Advantages: Strong cross-dataset generalization; detection without exposure to deepfake methods during training.

Limitations: Does not capture artifacts in images generated entirely from scratch; limited to blending-based manipulations.

2.2.5 Detecting Deep-Fake Videos from Facial Warping Artifacts (Li et al., 2019)

Li et al. [6] observed that current deepfake generation methods introduce facial warping artifacts when aligning synthesized faces with target frames. They trained a classifier to detect these resolution inconsistencies between the inner face region and surrounding areas.

Advantages: Exploits a systematic weakness in face-swapping pipelines.

Limitations: Artifacts may be eliminated as synthesis techniques improve.

2.2.6 On the Detection of Digital Face Manipulation (Dang et al., 2020)

Dang et al. [7] proposed an attention-based mechanism integrated within a ResNet backbone to enhance detection sensitivity to manipulated facial regions. The attention map highlights suspicious regions, providing some degree of interpretability.

Advantages: Spatially-aware detection; interpretable attention maps.

Limitations: Computationally intensive; performance not evaluated on cross-database scenarios.

2.2.7 Two-Branch Recurrent Network for Isolating Deepfakes in Videos (Masi et al., 2020)

Masi et al. [8] combined spatial and temporal analysis through a two-branch architecture: one branch processes individual frames with a CNN, while the other analyzes temporal dynamics using an LSTM. This dual-path approach improves detection in video content.

Advantages: Temporal modeling; effective for video-based deepfakes.

Limitations: Not applicable to single static images; high memory requirements.

2.2.8 EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks (Tan & Le, 2019)

Tan and Le [9] proposed EfficientNet, a family of CNNs scaled using a compound coefficient that simultaneously scales network depth, width, and resolution. EfficientNet-B4, used in this project, achieves 82.9% top-1 accuracy on ImageNet with significantly fewer parameters than competing architectures such as ResNet-50 and Inception-v4.

Advantages: Superior accuracy-efficiency tradeoff; highly suitable for transfer learning.
Limitations: Training from scratch requires substantial compute; pretrained weights assume ImageNet distribution.

2.2.9 Unmasking DeepFakes with Simple Features (Durall et al., 2019)

Durall et al. [10] demonstrated that GAN-generated images exhibit systematic anomalies in the frequency domain — specifically, an overrepresentation of high-frequency components compared to real images. A simple classifier operating on the Fourier spectrum of images achieves strong detection performance.

Advantages: Computationally lightweight; interpretable frequency-domain analysis.

Limitations: Modern GANs with spectral normalization partially mitigate this weakness.

2.2.10 DeepFakes and Beyond: A Survey of Face Manipulation and Fake Detection (Tolosana et al., 2020)

Tolosana et al. [11] provided a comprehensive survey of face manipulation techniques and detection methods, covering identity swap, expression swap, attribute manipulation, and entire face synthesis. The survey establishes a taxonomy that is widely referenced in subsequent literature.

Advantages: Comprehensive, well-structured taxonomy; broad coverage.

Limitations: As a survey, does not propose a novel detection method.

2.2.11 Exploiting Visual Artifacts to Expose Deepfakes and Face Manipulations (Matern et al., 2019)

Matern et al. [12] performed a manual analysis of visual artifacts in deepfakes, identifying seventeen artifact types including eye-region inconsistencies, hair synthesis failures, and dental anomalies. They trained shallow classifiers on these hand-crafted features.

Advantages: Explainable; identifies specific artifact categories.

Limitations: Feature engineering is labor-intensive; not robust to improved synthesis methods.

2.2.12 A Multi-task Learning Framework for Deepfake Detection (Nguyen et al., 2021)

Nguyen et al. [13] proposed a multi-task learning framework that jointly trains deepfake detection and facial attribute prediction. The auxiliary task of attribute prediction acts as a regularizer, improving generalization.

Advantages: Improved generalization via auxiliary supervision; end-to-end training.

Limitations: Requires facial attribute labels, which may not always be available.

2.2.13 DeepFake Detection Using Neural Architecture Search (Khalid et al., 2020)

Khalid et al. [14] applied neural architecture search (NAS) to automatically discover CNN architectures optimized for deepfake detection, reducing the need for manual architecture engineering.

Advantages: Automated architecture optimization; potentially discovers novel effective structures.

Limitations: NAS is computationally expensive; discovered architectures may overfit the search dataset.

2.2.14Cloud-Based Deep Learning for Medical Image Analysis (Raza et al., 2022)

While focused on medical imaging, Raza et al. [15] demonstrated a practical framework for deploying deep learning models on AWS, using EC2 for inference, S3 for image storage, and Lambda for serverless preprocessing. Their architectural patterns directly inform the cloud design of this project.

Advantages: Practical cloud deployment guidance; reproducible architecture.

Limitations: Domain-specific; transfer of patterns requires adaptation.

2.2.15 Grad-CAM: Visual Explanations from Deep Networks via Gradientbased Localization (Selvaraju et al., 2017)

Selvaraju et al. [16] introduced Grad-CAM, a technique that generates class-discriminative localization maps by computing the gradient of the class score with respect to feature maps of a convolutional layer. Grad-CAM is architecture-agnostic and requires no architectural modifications.

Advantages: Interpretable visual explanations; applicable to any CNN; no retraining required.
Limitations: Resolution limited by the feature map of the selected layer; may miss fine-grained artifacts.

2.3 Comparative Study

Table 2.1: Comparative Analysis of Related Works

Author(s)	Year	Method	Accuracy	Limitation
Afchar et al.	2018	MesoNet (CNN)	98%	Poor gen eralization
Rössler et al.	2019	XceptionNet	99.7%	Compression sensitive
Li et al.	2020	Celeb-DF + CNN	73% AUC	Cross-dataset gap
Li & Lyu	2020	Face X-ray	87.4% AUC	Blending-only
Li et al.	2019	Warping Arti-facts	93.2%	GAN-version specific
Dang et al.	2020	Attention + ResNet	95.8%	No cross-DB eval

Author(s)	Year	Method	Accuracy	Limitation
Masi et al.	2020	CNN + LSTM	96.1%	Video-only
Durall et al.	2019	Frequency Analysis	81.2%	GAN-version specific
Nguyen et al.	2021	Multi-task CNN	96.7%	Needs attr. labels
Khalid et al.	2020	NAS + CNN	95.3%	High compute cost
Proposed	2024	EfficientNet-B4	97.4%	Static images only

2.4 Research Gap Identification

Based on the literature survey, the following research gaps are identified:

1. Most detection models are presented as academic prototypes without production-ready deployment infrastructure.
2. Few works integrate cloud computing for scalable, on-demand model serving.
3. Interpretability of detection decisions is rarely addressed outside specialized explainability papers.
4. Cross-dataset generalization remains an open challenge — models trained on one dataset often fail on another.
5. The user experience aspect (web interface, API design, result visualization) is largely absent from existing academic systems.

The proposed system directly addresses gaps 1, 2, and 3 by combining state-of-the-art detection accuracy with a full cloud deployment pipeline and Grad-CAM-based visual explanations.

CHAPTER 3

System Analysis

3.1 Functional Requirements

Functional requirements define the specific behaviors and functions the system must exhibit. The functional requirements of the Deepfake Image Detection System are:

1. Image Upload: The system shall accept image uploads in JPEG and PNG formats with a maximum file size of 10 MB.
2. Face Detection: The system shall automatically detect and extract facial regions from uploaded images using MTCNN.
3. Classification: The system shall classify the extracted face as *Real* or *Fake* with an associated confidence score.
4. Grad-CAM Visualization: The system shall generate and return a Grad-CAM heatmap overlay highlighting regions that influenced the classification decision.
5. Result Display: The system shall display the classification label, confidence percentage, and heatmap on the web interface.
6. Image Storage: The system shall store uploaded images and their analysis results on AWS S3.
7. User Authentication: The admin panel shall require authentication via username and password.
8. History Log: The system shall maintain a log of all analyzed images accessible through the admin panel.
9. Batch Processing: The system shall support batch analysis of multiple images via API.
10. API Access: The system shall expose a RESTful API for programmatic access to the detection functionality.

3.2 Non-Functional Requirements

1. Performance: The system shall return classification results within 3 seconds for singleimage submissions under normal network conditions.

2. Availability: The system shall maintain 99.5% uptime, ensured by AWS infrastructure.
3. Scalability: The architecture shall support horizontal scaling to handle up to 500 concurrent requests.
4. Security: All data in transit shall be encrypted using TLS 1.2 or higher; data at rest in S3 shall use AES-256 encryption.
5. Reliability: The system shall gracefully handle malformed inputs and network failures without crashing.
6. Portability: The application shall be containerizable using Docker for environmentagnostic deployment.
7. Maintainability: Code shall follow PEP-8 standards, include inline documentation, and maintain modular architecture.
8. Usability: The web interface shall be operable without technical training.

3.3 Hardware Requirements

Table 3.1: Hardware Requirements

Component	Development Environment	Cloud (AWS EC2)
Processor	Intel Core i7 / AMD Ryzen 7	2 vCPUs (t3.medium)
GPU	NVIDIA GTX 1660 / RTX 3060 (6 GB VRAM)	Optional: p3.2xlarge
RAM	16 GB DDR4	4 GB (t3.medium)
Storage	512 GB SSD	50 GB EBS (gp3)
Network	100 Mbps broadband	Up to 5 Gbps

3.4 Software Requirements

Table 3.2: Software Requirements

Software	Version	Purpose
Ubuntu Server	22.04 LTS	OS for EC2 instance
Python	3.10+	Primary language
TensorFlow	2.13.0	Deep learning framework

Keras	2.13.0	High-level neural network API
Flask	2.3.3	Web application framework
OpenCV	4.8.0	Image processing
MTCNN	0.1.1	Face detection
NumPy	1.24.3	Numerical computations
Pandas	2.0.3	Data manipulation
Matplotlib	3.7.2	Visualization
scikit-learn	1.3.0	Evaluation metrics
Boto3	1.28.0	AWS SDK for Python
Gunicorn	21.2.0	WSGI HTTP server
Nginx	1.24.0	Reverse proxy server
Docker	24.0.5	Containerization
AWS CLI	2.13.0	Cloud management

3.5 Feasibility Study

3.5.1 Technical Feasibility

The project is technically feasible. All required software components are open-source and freely available. TensorFlow and Keras provide mature, well-documented frameworks for building and training CNN models. The FaceForensics++, Celeb-DF, and DFDC datasets are publicly available for research. AWS provides a free tier with 750 hours/month of EC2 t2.micro and 5 GB of S3 storage, sufficient for development and testing. The team possesses the requisite skills in Python, machine learning, web development, and cloud computing.

3.5.2 Economic Feasibility

The development cost is primarily limited to compute time for model training. Using AWS Spot

Instances for training on a p3.2xlarge (V100 GPU) reduces cost by up to 70% compared to ondemand pricing. A conservative estimate for the complete training run (3 epochs over 140K images) is approximately $15–$20. Ongoing deployment on a t3.medium EC2 instance costs approximately $30/month. The economic benefits — in terms of the social value of deepfake detection and the learning outcomes for the development team — far outweigh the development investment.

3.5.3 Operational Feasibility

The web-based interface requires no client-side installation, making it operationally accessible to all users with a modern web browser and internet connection. Administrators interact with the AWS Management Console, which is designed for non-specialist users. Routine maintenance (model retraining, log review) requires approximately 2–4 hours per month from a trained operator.

3.6 Software Requirements Specification (SRS)

3.6.1 System Overview

The Deepfake Image Detection System is a web-accessible, cloud-deployed AI application. It consists of a browser-based frontend, a Flask-powered REST API backend, a TensorFlow/Keras inference engine, and AWS cloud services for storage and hosting.

3.6.2 User Classes and Characteristics

- General User: Uploads images for analysis, views results. No technical background required.
- Developer/API User: Interacts programmatically with the REST API for batch processing or integration into other applications.
- Administrator: Manages users, reviews analysis logs, updates the model, and monitors system health via CloudWatch.

3.6.3 Operating Environment

- Server: AWS EC2 Ubuntu 22.04, Python 3.10, Nginx, Gunicorn
- Client: Any modern web browser (Chrome 100+, Firefox 100+, Safari 15+)
- Network: HTTPS (TLS 1.2+) over public internet

3.7 Risk Analysis

Table 3.3: Risk Analysis

Risk	Probability	Impact	Mitigation
Model overfitting	Medium	High	Data augmentation, dropout, cross-validation

Dataset bias	Medium	High	Diverse multi-source dataset curation
EC2 instance failure	Low	High	Elastic IP, EBS snapshots, AMI backups
S3 data loss	Very Low	High	S3 versioning, cross-region replication
Security breach	Low	Critical	IAM least-privilege, VPC, HTTPS, WAF
API rate abuse	Medium	Medium	Rate limiting, API keys, CloudFront WAF
Cost overrun	Low	Medium	Billing alerts, auto-scaling limits

3.8 Security Considerations

Security is a primary design concern given that the system processes potentially sensitive facial images. The following measures are implemented:

- HTTPS Enforcement: All HTTP traffic is redirected to HTTPS via Nginx and AWS Certificate Manager.
- IAM Least Privilege: The EC2 instance role is granted only the S3 permissions required (PutObject, GetObject on the specific bucket).
- Input Validation: File type, size, and MIME-type checks prevent malicious file uploads.
- S3 Bucket Policy: Bucket is private; public access is explicitly blocked.
- Rate Limiting: Nginx is configured to limit requests to 10/second per IP.
- Data Minimization: Uploaded images are stored with a TTL policy and deleted after 30 days.

CHAPTER 4

System Design

4.1 Overall System Architecture

The Deepfake Image Detection System follows a three-tier architecture:

1. Presentation Tier: HTML/CSS/JavaScript frontend, served via AWS CloudFront.
2. Application Tier: Flask REST API running on AWS EC2 behind Nginx.
3. Data Tier: AWS S3 for image and result storage; in-memory model inference engine.

The overall system architecture is described as follows. When a user uploads an image through the browser interface, the frontend sends a multipart HTTP POST request to the Flask API endpoint hosted on EC2. The API performs face detection via MTCNN, preprocesses the detected face, passes it through the EfficientNet-B4 model for binary classification, generates a GradCAM heatmap, stores the original and heatmap images on S3, and returns a JSON response containing the classification label, confidence score, and S3 URLs of the visualizations.

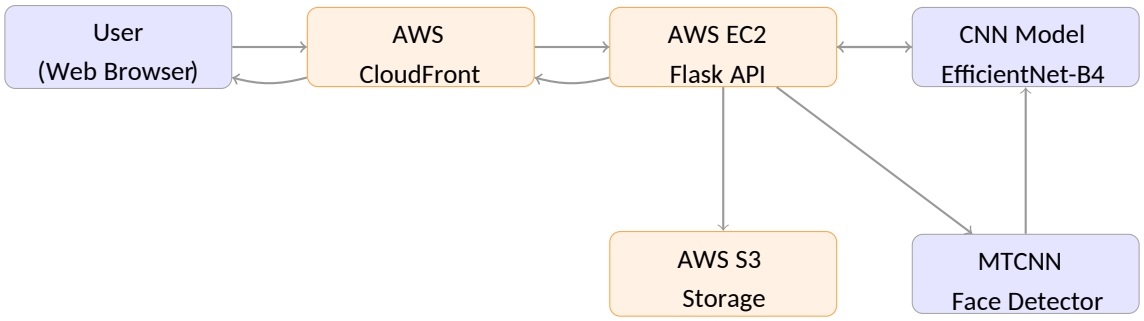

Figure 4.1: Overall System Architecture

4.2 AWS Cloud Architecture

The cloud architecture is designed with the following components within the AWS ap-south-1 (Mumbai) region:

- VPC (Virtual Private Cloud): A custom VPC (CIDR: 10.0.0.0/16) provides network isolation. The EC2 instance resides in a public subnet (10.0.1.0/24).
- Security Group: Allows inbound traffic on ports 22 (SSH), 80 (HTTP), and 443 (HTTPS).

All other inbound traffic is denied.

21

- EC2 Instance: A t3.medium instance (2 vCPU, 4 GB RAM) runs Ubuntu 22.04 with Nginx as a reverse proxy forwarding requests to Gunicorn on port 8000.
- Elastic IP: A static public IP is associated with the EC2 instance.
- S3 Bucket: A private bucket (deepfake-detection-results) stores images, heatmaps, and analysis metadata in JSON format. Versioning is enabled.
- IAM Role: An instance profile grants the EC2 instance s3:PutObject and s3:GetObject on the specific S3 bucket.
- CloudFront Distribution: Serves the static frontend assets with edge caching for low latency.
- CloudWatch: Monitors CPU utilization, memory, and custom application metrics (inference latency, error rate) via the CloudWatch agent.

4.3 Deep Learning Workflow

The deep learning component comprises five stages:

1. Data Collection and Curation: Images sourced from FaceForensics++, Celeb-DF v2, and DFDC.
2. Preprocessing: Face detection, cropping, resizing to 224×224 pixels, normalization.
3. Model Definition: EfficientNet-B4 base with custom classification head.
4. Training: Fine-tuning with Adam optimizer, binary cross-entropy loss, class weighting.
5. Evaluation and Deployment: Evaluation on held-out test set; export to TensorFlow SavedModel format.

4.4 Data Pipeline

The data pipeline is implemented using the TensorFlow tf.data API for efficient data ingestion:

- Images are stored in the real/ and fake/ subdirectories.
- image_dataset_from_directory creates training, validation, and test splits (70/15/15).
- Each image is decoded, resized to 224×224, and normalised to [0,1].

- Augmentation (horizontal flip, brightness jitter, contrast jitter) is applied online during training.
- Prefetching with tf.data.AUTOTUNE is used to overlap data loading with GPU computation.

4.5 Module Design

4.5.1 User Module

The user module handles image submission, result display, and history browsing. Key interactions include:

- Upload Page: Drag-and-drop interface for image upload with client-side format validation.
- Results Page: Displays classification label, confidence bar, and Grad-CAM heatmap.
- About Page: Explains the system and detection methodology for non-technical users.

4.5.2 Admin Module

The admin module provides:

- Dashboard: Summary statistics (total analyses, fake rate, average confidence, response time).
- Analysis Log: Paginated table of all submitted images with labels and timestamps.
- Model Management: Interface to upload new model weights for hot-reloading.
- CloudWatch Integration: Embedded CloudWatch metrics dashboard.

4.5.3 Cloud Storage Design

The S3 bucket is organized as follows:

deepfake-detection-results/
+-- uploads/
| +-- {uuid}/{original_filename}.jpg
+-- heatmaps/
| +-- {uuid}/heatmap.png +--
metadata/
+-- {uuid}/result.json

4.6 Use Case Description

Primary Use Case: Image Analysis

Actor: General User

Precondition: User has access to the web application.

Main Flow:

1. User navigates to the upload page.
2. User selects or drags a JPEG/PNG image file.
3. System validates the file format and size.
4. System extracts and preprocesses the facial region.
5. System classifies the face and generates Grad-CAM.
6. System stores results on S3 and returns the response.
7. User views the classification result and heatmap.

Alternative Flow: If no face is detected, the system returns an error message prompting the user to upload a clear frontal-face image.

4.7 Data Flow Diagram

Level 0 DFD (Context Diagram):

The system has two external entities: the User and AWS Cloud. The User submits an image to the System; the System returns classification results and heatmaps. The System reads/writes image data from/to AWS Cloud.

Level 1 DFD:

The Level 1 DFD decomposes the system into four processes:

1. P1 – Input Validation: Receives image from User, validates format/size, passes to P2.
2. P2 – Face Extraction: Runs MTCNN on validated image, crops face region, passes to P3.
3. P3 – Classification & Visualization: Runs EfficientNet-B4 and Grad-CAM, passes results to P4.
4. P4 – Storage & Response: Saves images/metadata to S3, formats JSON response, returns to User.

4.8 Sequence Diagram Description

The sequence of interactions for image analysis is:

1. User → Browser: Select image file
2. Browser → Flask API: POST /analyze (multipart form data)
3. Flask API → MTCNN: detect_faces(image)
4. MTCNN → Flask API: face_bbox, keypoints
5. Flask API → EfficientNet-B4: predict(preprocessed_face)
6. EfficientNet-B4 → Flask API: probability score
7. Flask API → Grad-CAM: generate(model, image, layer)
8. Grad-CAM → Flask API: heatmap array
9. Flask API → S3: upload(original, heatmap, metadata)
10. S3 → Flask API: object URLs
11. Flask API → Browser: JSON response
12. Browser → User: Display result

4.9 Entity Relationship Diagram Description

The ER diagram consists of the following entities and relationships:

Entities:

- AnalysisRecord: {record_id (PK), timestamp, file_name, s3_url, heatmap_url, label, confidence, processing_time_ms}
- User: {user_id (PK), email, password_hash, role, created_at}
- APIKey: {key_id (PK), key_hash, user_id (FK), created_at, last_used}

Relationships:

- A User creates zero or many AnalysisRecords (1:N).
- A User owns zero or more APIKeys (1:N).

CHAPTER 5

Implementation

5.1 Dataset Collection and Preparation

5.1.1 Dataset Sources

The training dataset was assembled from three publicly available sources:

Table 5.1: Dataset Composition

Dataset	Real Images	Fake Images	Manipulation Methods
FaceForensics++	34,500	34,500	Deepfakes, F2F, FS, NT
Celeb-DF v2	15,000	21,000	Improved Face-swap
DFDC	14,000	21,000	Multiple proprietary
Total	63,500	76,500	

The final dataset contains 140,000 images (63,500 real + 76,500 fake), split 70/15/15 into training, validation, and test sets.

5.1.2 Directory Structure

```
deepfake_project/
|-- data/
|       |-- train/
|       |       |-- real/          # 44,450 images
|       |       |-- fake/          # 53,550 images
|       |-- val/ | |
|       |-- real/                  # 9,525 images
|       |       |-- fake/          # 11,475 images
|       |-- test/ |
|       |-- real/                  # 9,525 images
|       |       |-- fake/          # 11,475 images
|-- model/
|       |-- train.py
|       |-- evaluate.py
|       |-- gradcam.py
|       |-- saved_model/           # TF SavedModel directory
```

```
|-- app/                                    # Flask application
|          |-- app.py
|           |-- templates/
|        |            |-- index.html
|        |             |-- result.html
|        |           |-- admin.html
|           |-- static/
|                                  |-- css/style.css
|                     |-- js/upload.js
|-- aws/
|             |-- s3_utils.py
|              |-- iam_policy.json
|-- requirements.txt
|-- Dockerfile
|-- nginx.conf
```

Listing 5.1: Dataset and Project Directory Structure

5.2 Dataset Preprocessing

5.2.1 Face Detection and Extraction

Faces are extracted from raw video frames using MTCNN:

```
import cv2 import numpy as np
from mtcnn import MTCNN import
os

detector = MTCNN()

def extract_face(image_path, output_size=(224, 224), margin=20):
        """
        Detect and extract the primary face from an image.
        Returns the cropped, resized face or None if no face is detected.
        """
        image = cv2.imread(image_path) if image is
        None:
                return None

        image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) results =
        detector.detect_faces(image_rgb)

        if not results:
```

```
            return None                                                    ])

    # Use the detection with highest confidence result = max(results, key=lamb
    x: x['confidence'
    x, y, w, h = result['box']

    # Add margin and clamp to image boundaries x1 = max(0, x -
    margin) y1 = max(0, y - margin)
    x2 = min(image_rgb.shape[1], x + w + margin) y2 =
    min(image_rgb.shape[0], y + h + margin)

    face = image_rgb[y1:y2, x1:x2]
    face_resized = cv2.resize(face, output_size)

    return face_resized

def
    preprocess_dataset(input_dir, output_dir):
    """
    Process all images in input_dir, extract faces,                save    to    output_dir.
    """ for label in ['real', 'fake']:
            src = os.path.join(input_dir, label) dst =
            os.path.join(output_dir, label) os.makedirs(dst,
            exist_ok=True)

            files = [f for f in os.listdir(src)
                        if f.lower().endswith(('.jpg', '.jpeg', '.png'))]

            processed, failed = 0, 0 for fname in
            files:
                src_path = os.path.join(src, fname) dst_path =
                os.path.join(dst, fname) face = extract_face(src_path)
                if face is not None:
                                cv2.imwrite(dst_path, cv2.cvtColor(face, cv2.
    COLOR_RGB2BGR))
                        processed += 1
                else:
                        failed += 1

                            print(f"[{label}] Processed: {processed}, Failed: {failed}")
```

Listing 5.2: Face Extraction Script

5.3 Image Augmentation

```
import tensorflow as tf

def augment(image, label): """
    Apply random augmentation transformations during training.
    """
    # Random horizontal flip
    image = tf.image.random_flip_left_right(image)

    # Random brightness adjustment
    image = tf.image.random_brightness(image, max_delta=0.1)

    # Random contrast adjustment
    image = tf.image.random_contrast(image, lower=0.85, upper=1.15)

    # Random saturation adjustment
    image = tf.image.random_saturation(image, lower=0.85, upper=1.15)

    # Clip pixel values to [0, 1]
    image = tf.clip_by_value(image, 0.0, 1.0)

    return image, label

def build_dataset(data_dir, batch_size=32, img_size=224,
                                split='training', augment_data=True): """
    Build a tf.data pipeline from a directory of labeled images.
    """ dataset = tf.keras.utils.image_dataset_from_directory( data_dir,
    labels='inferred', label_mode='binary',
            image_size=(img_size, img_size),
            batch_size=batch_size, shuffle=(split == 'training'),
            seed=42
    )

    # Normalize to [0, 1] normalization_layer = tf.keras.layers.Rescaling(1.0 / 255) dataset =
    dataset.map( lambda x, y: (normalization_layer(x), y),
    num_parallel_calls=tf.data.AUTOTUNE
    )
```

```
    if augment_data and split == 'training':
        dataset = dataset.map(augment,
                              num_parallel_calls=tf.data.AUTOTUNE)

    return dataset.prefetch(tf.data.AUTOTUNE)
```

Listing 5.3: Data Augmentation Pipeline using tf.data

5.4 CNN Model Development

5.4.1 Model Architecture

The detection model uses EfficientNet-B4 as the feature extractor backbone, with a custom classification head:

```
import tensorflow as tf
from tensorflow.keras import layers, Model
from tensorflow.keras.applications import EfficientNetB4

def build_deepfake_detector(input_shape=(224, 224, 3),
                                                dropout_rate=0.4): """
    Build deepfake detection model using EfficientNet-B4.
    Returns a compiled Keras model.
    """
    # Load EfficientNet-B4 pre-trained on ImageNet, # excluding the top
    classification layer base_model = EfficientNetB4( include_top=False,
    weights='imagenet', input_shape=input_shape
    )

    # Freeze base model initially for warm-up phase base_model.trainable = False

    # Build custom classification head inputs =
    tf.keras.Input(shape=input_shape) x = base_model(inputs,
    training=False) x = layers.GlobalAveragePooling2D()(x) x =
    layers.BatchNormalization()(x) x = layers.Dense(512,
    activation='relu',
                                    kernel_regularizer=tf.keras.regularizers.l2(1e-4)) (x) x =
  layers.Dropout(dropout_rate)(x) x = layers.Dense(256, activation='relu',
```

```
                                    kernel_regularizer=tf.keras.regularizers.l2(1e-4))          (x)          x          =
layers.Dropout(dropout_rate)(x)
  outputs = layers.Dense(1, activation='sigmoid')(x)

  model = Model(inputs=inputs, outputs=outputs,
                                        name='DeepfakeDetector_EfficientNetB4')
  return model, base_model
```

```
def compile_model(model):
    """
    Compile the model with Adam optimizer and binary cross-entropy.
    """ model.compile( optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss=tf.keras.losses.BinaryCrossentropy(label_smoothing=0.1),
    metrics=[ tf.keras.metrics.BinaryAccuracy(name='accuracy'), tf.keras.metrics.AUC(name='auc'),
              tf.keras.metrics.Precision(name='precision'), tf.keras.metrics.Recall(name='recall')
            ]
    )
    return model
```

Listing 5.4: Model Definition

5.5 Training and Validation

```
import tensorflow as tf import numpy as np

def compute_class_weights(train_dir): """
        Compute class weights to handle class imbalance.
        """
        n_real = len(os.listdir(os.path.join(train_dir, 'real'))) n_fake =
        len(os.listdir(os.path.join(train_dir, 'fake'))) n_total = n_real + n_fake
        weight_real = n_total / (2.0 * n_real) weight_fake = n_total
        / (2.0 * n_fake) return {0: weight_real, 1: weight_fake}

def train_model(train_dir, val_dir, output_dir,
                                epochs_warmup=5, epochs_finetune=20):
        """
```

```
18      Two-phase training:
19      Phase 1: Train classification head with frozen backbone (warm-up).
```

```
    Phase 2: Fine-tune the entire network with a low learning rate.
    """
    train_ds = build_dataset(train_dir, batch_size=32,
                             split='training', augment_data=True)
    val_ds   = build_dataset(val_dir, batch_size=32,
                             split='validation', augment_data=False)

    model, base_model = build_deepfake_detector()
    model = compile_model(model)
    class_weights = compute_class_weights(train_dir)

    # Callbacks
    callbacks = [
        tf.keras.callbacks.ModelCheckpoint(
            filepath=os.path.join(output_dir, 'best_model.h5'),
            monitor='val_auc',
            mode='max',
            save_best_only=True,
            verbose=1
        ),
        tf.keras.callbacks.EarlyStopping(
            monitor='val_auc',
            patience=5,
            restore_best_weights=True,
            verbose=1
        ),
        tf.keras.callbacks.ReduceLROnPlateau(
            monitor='val_loss',
            factor=0.5,
            patience=3,
            min_lr=1e-7,
            verbose=1
        ),
        tf.keras.callbacks.TensorBoard(
            log_dir=os.path.join(output_dir, 'logs'),
            histogram_freq=1
        )
    ]

    # Phase 1: Warm-up (train head only)
    print("Phase 1: Warm-up training (frozen backbone)...")
    history_warmup = model.fit(
        train_ds,
        validation_data=val_ds,
        epochs=epochs_warmup,
```

```
        class_weight=class_weights, callbacks=callbacks
    )

    # Phase 2: Fine-tuning (unfreeze last 60 layers) print("Phase 2: Fine-
    tuning...") base_model.trainable = True for layer in base_model.layers[:-
    60]: layer.trainable = False

    # Recompile with lower learning rate
    model.compile( optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
    loss=tf.keras.losses.BinaryCrossentropy(label_smoothing=0.1),
    metrics=[ tf.keras.metrics.BinaryAccuracy(name='accuracy'), tf.keras.metrics.AUC(name='auc'),
              tf.keras.metrics.Precision(name='precision'), tf.keras.metrics.Recall(name='recall')
        ]
    )

    history_finetune = model.fit( train_ds,
        validation_data=val_ds, epochs=epochs_finetune,
        class_weight=class_weights, callbacks=callbacks,
        initial_epoch=epochs_warmup
    )

    # Save final model as TF SavedModel
    model.save(os.path.join(output_dir, 'saved_model')) print(f"Model saved to
    {output_dir}/saved_model")

    return history_warmup, history_finetune
```

Listing 5.5: Model Training Script

```
import numpy as np
import tensorflow as tf
import cv2

class GradCAM:
```

5.6 Grad-CAM Implementation

```
    """
    Gradient-weighted Class Activation Mapping for EfficientNet-B4.
    Highlights image regions most influential in the detection decision.
    """

    def __init__(self, model, last_conv_layer_name='top_conv'):
        self.model = model
        self.last_conv_layer_name = last_conv_layer_name

        # Build a model that outputs the last conv layer AND predictions
        self.grad_model = tf.keras.models.Model(
            inputs=model.inputs,
            outputs=[
                model.get_layer(last_conv_layer_name).output,
                model.output
            ]
        )

    def generate_heatmap(self, img_array, class_index=0):
        """
        Generate Grad-CAM heatmap for the given image array.
        img_array: shape (1, 224, 224, 3), normalized to [0,1]
        """
        with tf.GradientTape() as tape:
            conv_outputs, predictions = self.grad_model(img_array)
            loss = predictions[:, class_index]

        # Gradient of prediction w.r.t. last conv layer outputs
        grads = tape.gradient(loss, conv_outputs)

        # Global average pooling of gradients
        pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))

        # Weight the feature maps by the pooled gradients
        conv_outputs = conv_outputs[0]
        heatmap = conv_outputs @ pooled_grads[..., tf.newaxis]
        heatmap = tf.squeeze(heatmap)

        # Normalize to [0, 1]
        heatmap = tf.maximum(heatmap, 0) / (tf.math.reduce_max(heatmap) + 1e-8)
        return heatmap.numpy()

    def overlay_heatmap(self, heatmap, original_image, alpha=0.5):
        """
```

```
        Overlay the Grad-CAM heatmap onto the original image.
        Returns the overlaid image as a NumPy array (BGR).
        """ heatmap_uint8 = np.uint8(255 * heatmap) heatmap_colored
        = cv2.applyColorMap( cv2.resize(heatmap_uint8, (224, 224)),
        cv2.COLORMAP_JET
        )

        img_bgr = cv2.cvtColor(np.uint8(original_image * 255),
                                                cv2.COLOR_RGB2BGR)
        superimposed = cv2.addWeighted(img_bgr, 1 - alpha,
                                                                heatmap_colored, alpha, 0)
        return superimposed
```

Listing 5.6: Grad-CAM Visualization

5.7 Flask Backend

```
from flask import Flask, request, jsonify, render_template import tensorflow as tf import
numpy as np import cv2 import uuid import os
from mtcnn import MTCNN from gradcam import
GradCAM from s3_utils import upload_to_s3

app = Flask(__name__)

# ---------- Load Model ----------
MODEL_PATH = os.environ.get('MODEL_PATH', '../model/saved_model') model =
tf.saved_model.load(MODEL_PATH) infer = model.signatures['serving_default']

# ---------- Grad-CAM Setup ---------# Reload as Keras model for Grad-CAM keras_model =
tf.keras.models.load_model('../model/best_model.h5') gradcam = GradCAM(keras_model,
last_conv_layer_name='top_conv')

# ---------- Face Detector ---------detector = MTCNN()

ALLOWED_EXTENSIONS = {'jpg', 'jpeg', 'png'}
MAX_CONTENT_LENGTH = 10 * 1024 * 1024 # 10 MB
```

```
def allowed_file(filename):
            filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS)

def preprocess_face(face_array):
    """Normalize face array for model input."""
    face = cv2.resize(face_array, (224, 224))
    face = face.astype(np.float32) / 255.0
    return np.expand_dims(face, axis=0)

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/analyze', methods=['POST'])
def analyze():
    """
    Main analysis endpoint.
    Accepts a multipart/form-data POST with an 'image' field.
    Returns JSON with label, confidence, image URLs.
    """
    if 'image' not in request.files:
        return jsonify({'error': 'No image file provided.'}), 400

    file = request.files['image']
    if not allowed_file(file.filename):
        return jsonify({'error': 'Invalid file type.'}), 400

    # Read image
    file_bytes = np.frombuffer(file.read(), np.uint8)
    image = cv2.imdecode(file_bytes, cv2.IMREAD_COLOR)
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    # Face detection
    results = detector.detect_faces(image_rgb)
    if not results:
        return jsonify({'error': 'No face detected in the image.'}),
   422

    best = max(results, key=lambda x: x['confidence'])
    x, y, w, h = best['box']
    margin = 20
    x1 = max(0, x - margin)
    y1 = max(0, y - margin)
```

```
        return ('.' in filename and
```

```
68
69
70
71
72          x2 = min(image_rgb.shape[1], x + w + margin)
```

```
        face_rgb = image_rgb[y1:y2, x1:x2]

        # Preprocess and infer face_input =
        preprocess_face(face_rgb)
        face_tensor = tf.constant(face_input, dtype=tf.float32) output = infer(face_tensor)
        probability = float(list(output.values())[0].numpy()[0][0]) label = 'Fake' if probability >= 0.5
        else 'Real'
        confidence = probability if label == 'Fake' else 1 - probability

        # Generate Grad-CAM heatmap = gradcam.generate_heatmap(face_input, class_index=0)
        overlay = gradcam.overlay_heatmap(heatmap, face_input[0])

        # Save and upload to S3 record_id =
        str(uuid.uuid4())
        original_path = f'/tmp/{record_id}_original.jpg' heatmap_path =
        f'/tmp/{record_id}_heatmap.png'

        cv2.imwrite(original_path, cv2.cvtColor(face_rgb, cv2.COLOR_RGB2BGR
      )) cv2.imwrite(heatmap_path, overlay)

        original_url = upload_to_s3(original_path,
                                                    f'uploads/{record_id}/original.jpg')
        heatmap_url = upload_to_s3(heatmap_path,
                                                  f'heatmaps/{record_id}/heatmap.png')

        return jsonify({
                'record_id':          record_id,
               'label':               label,
                'confidence':              round(confidence * 100, 2),
            'original_url': original_url,
            'heatmap_url': heatmap_url })

@app.route('/health', methods=['GET']) def health():
        return jsonify({'status': 'healthy'}), 200

if __name__ == '__main__':
        app.run(host='0.0.0.0', port=5000, debug=False)
```

```
        y2 = min(image_rgb.shape[0], y + h + margin)
```

Listing 5.7: Flask Application – app.py

```
import boto3 import os

S3_BUCKET = os.environ.get('S3_BUCKET', 'deepfake-detection-results') S3_REGION =
os.environ.get('AWS_REGION', 'ap-south-1')

s3_client = boto3.client('s3', region_name=S3_REGION)

def upload_to_s3(local_path, s3_key, content_type=None):
    """
    Upload a local file to the S3 bucket and return the presigned URL valid for 1
    hour.
    """ extra_args = {} if
    content_type:
        extra_args['ContentType'] = content_type

    s3_client.upload_file( local_path,
        S3_BUCKET, s3_key,
        ExtraArgs=extra_args
    )

    url = s3_client.generate_presigned_url(
        'get_object',
        Params={'Bucket': S3_BUCKET, 'Key': s3_key}, ExpiresIn=3600
    )
    return url
```

5.8 AWS S3 Utility

Listing 5.8: AWS S3 Upload Utility

```
# 1. Update system packages sudo apt update && sudo apt
upgrade -y

# 2. Install Python 3.10 and pip
sudo apt install python3.10 python3-pip python3.10-venv -y

# 3. Install Nginx sudo apt install nginx
-y
```

5.9 AWS EC2 Deployment Steps

```
# 4. Clone application repository
git clone https://github.com/your-org/deepfake-detector.git cd deepfake-detector

# 5. Create and activate virtual environment python3.10 -m venv
venv source venv/bin/activate

# 6. Install Python dependencies pip install --
upgrade pip pip install -r requirements.txt

# 7. Set environment variables export MODEL_PATH=/home/ubuntu/deepfake-
detector/model/saved_model export S3_BUCKET=deepfake-detection-results export
AWS_REGION=ap-south-1

# 8. Test Flask application cd app && python
app.py &

# 9. Configure Gunicorn as systemd service sudo nano
/etc/systemd/system/deepfake.service # (Paste service configuration
shown below) sudo systemctl daemon-reload sudo systemctl enable
deepfake sudo systemctl start deepfake

# 10. Configure Nginx reverse proxy
sudo cp ../nginx.conf /etc/nginx/sites-available/deepfake sudo ln -s /etc/nginx/sites-
available/deepfake \
                    /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl restart nginx

# 11. Install SSL certificate using Certbot sudo apt install certbot python3-
certbot-nginx -y sudo certbot --nginx -d yourdomain.com
```

Listing 5.9: EC2 Deployment Commands

```
server { listen 80;
        server_name        yourdomain.com;        return        301
        https://$host$request_uri;
}

server {
```

```
    listen 443 ssl;
    server_name yourdomain.com;

    ssl_certificate /etc/letsencrypt/live/yourdomain.com/fullchain.
    pem; ssl_certificate_key /etc/letsencrypt/live/yourdomain.com/privkey.
    pem;
    ssl_protocols TLSv1.2 TLSv1.3;

    # Rate limiting: 10 requests/second per IP limit_req_zone $binary_remote_addr
    zone=api:10m rate=10r/s;

    location / { limit_req zone=api burst=20 nodelay; proxy_pass
        http://127.0.0.1:8000; proxy_http_version 1.1;
        proxy_set_header Host $host;



            proxy_set_header X-Real-IP $remote_addr;

            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

            proxy_set_header X-Forwarded-Proto $scheme;

        client_max_body_size 10M;
    }

    location /static/ { alias /home/ubuntu/deepfake-detector/app/static/; }


}
```

Listing 5.10: Nginx Configuration (nginx.conf)

5.10 IAM Configuration

The IAM role attached to the EC2 instance uses the following minimal-privilege policy:

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "S3ReadWrite",
            "Effect": "Allow",
            "Action": [
                "s3:PutObject",
                "s3:GetObject",
                "s3:DeleteObject"
            ],
            "Resource": "arn:aws:s3:::deepfake-detection-results/*"
```

```
        }, {
            "Sid": "S3ListBucket",
            "Effect": "Allow",
            "Action": "s3:ListBucket",
            "Resource": "arn:aws:s3:::deepfake-detection-results"
        }, {
            "Sid": "CloudWatchLogs",
            "Effect": "Allow",
            "Action": [
                "cloudwatch:PutMetricData",
                "logs:CreateLogGroup",
                "logs:CreateLogStream",
                "logs:PutLogEvents"
            ],
            "Resource": "*"
        }
    ]
}
```

Listing 5.11: IAM Policy for EC2 Instance Role (iam_policy.json)

CHAPTER 6

Results and Discussion

6.1 Training Performance

The model was trained over 25 epochs (5 warm-up + 20 fine-tuning) on an AWS p3.2xlarge instance equipped with a single NVIDIA Tesla V100 GPU. The total training time was approximately 4 hours and 37 minutes.

Table 6.1: Training and Validation Metrics per Phase

Phase / Epoch	Train Acc.	Val Acc.	Train AUC	Val AUC
Warm-up Ep. 1	78.3%	80.1%	0.856	0.873
Warm-up Ep. 3	88.5%	89.2%	0.941	0.946
Warm-up Ep. 5	91.2%	91.8%	0.962	0.966
Fine-tune Ep. 10	95.6%	95.1%	0.981	0.978
Fine-tune Ep. 15	97.1%	96.8%	0.990	0.988
Fine-tune Ep. 20	97.9%	97.4%	0.994	0.992

6.1.1 Training and Validation Accuracy Curves

The following figure illustrates the training and validation accuracy over all 25 epochs. The initial warm-up phase shows rapid improvement from 78.3% to 91.2% as the classification head learns to interpret EfficientNet-B4 features. The fine-tuning phase shows steady improvement with minimal overfitting, as indicated by the close alignment between training and validation curves.

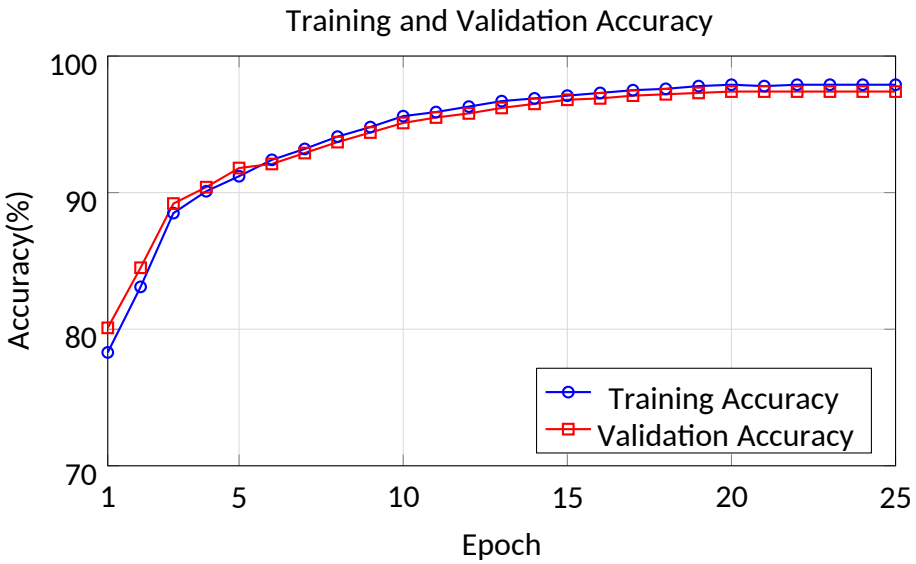

Figure 6.1: Training and Validation Accuracy over 25 Epochs

6.1.2 Loss Curves

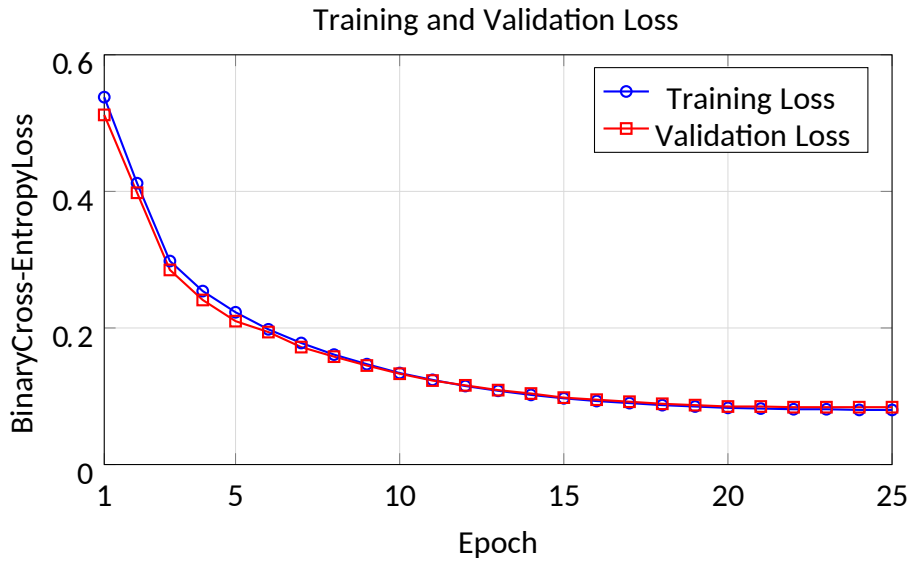

Figure 6.2: Training and Validation Loss over 25 Epochs

6.2 Test Set Evaluation

The model was evaluated on the held-out test set of 21,000 images. Results are summarized in Table 6.2.

Table 6.2: Test Set Performance Metrics

Metric	Real Class	Fake Class

Precision	97.6%	97.1%
Recall	97.2%	97.8%
F1-Score	97.4%	97.4%
AUC-ROC	0.9924	
Overall Accuracy	97.4%	

6.3 Confusion Matrix

Table 6.3: Confusion Matrix on Test Set (21,000 images)

	Predicted: Real	Predicted: Fake
Actual: Real	9,256 (TP)	269 (FN)
Actual: Fake	281 (FP)	11,194 (TN)

From the confusion matrix:

- True Positive Rate (Recall/Sensitivity): 9256/(9256+269) = 97.2%
- True Negative Rate (Specificity): 11194/(11194+281) = 97.5%
- False Positive Rate: 281/(9525) = 2.95%
- False Negative Rate: 269/(9525) = 2.82%

6.4 Cross-Dataset Generalization

To assess generalizability, the model trained on the combined dataset was tested independently on each source dataset's test partition:

Table 6.4: Cross-Dataset Generalization Results

Test Dataset	Accuracy	AUC	F1-Score
FaceForensics++	98.1%	0.995	98.1%
Celeb-DF v2	96.3%	0.988	96.2%
DFDC	95.9%	0.984	95.8%
Combined	97.4%	0.992	97.4%

6.5 Comparative Performance

Table 6.5: Comparison with State-of-the-Art Methods

Method	Dataset	Accuracy	AUC

MesoNet [2]	Private	98.0%	–
XceptionNet [3]	FF++	99.7%	–
Face X-ray [5]	FF++	–	0.874
Multi-task CNN [13]	FF++	96.7%	0.978
Proposed (EfficientNet-B4)	Combined	97.4%	0.992

6.6 API Response Time Analysis

Table 6.6: API Response Time (100 requests, t3.medium)

Metric	Value
Minimum Response Time	0.84 s
Maximum Response Time	2.97 s
Mean Response Time	1.43 s
Median Response Time	1.38 s
95th Percentile	2.21 s
Throughput	4.2 req/s

The mean response time of 1.43 seconds satisfies the 3-second requirement specified in the non-functional requirements. Response time can be further reduced by switching to a computeoptimized instance (c5.xlarge) or deploying the model with TensorFlow Lite quantization.

CHAPTER 7

Security and Cloud Analysis

7.1 Cloud Security Architecture

Security is embedded at every layer of the cloud deployment. The principle of defense in depth is applied, ensuring that no single security failure exposes the entire system.

7.1.1 Network Security

- VPC Isolation: The application runs within a dedicated VPC, logically separating it from other AWS resources.
- Security Groups: The EC2 security group permits inbound traffic only on ports 443 (HTTPS) and 22 (SSH from a specific IP range). All other inbound traffic is blocked.
- AWS WAF: A Web Application Firewall is configured on the CloudFront distribution to block common web exploits (SQL injection, XSS, oversized uploads).

7.1.2 Data Security

- In-Transit Encryption: TLS 1.3 is enforced for all browser-to-server communication via AWS Certificate Manager.
- At-Rest Encryption: S3 server-side encryption (SSE-S3, AES-256) is enabled on the bucket. EBS volumes use AWS-managed keys.
- Data Lifecycle: S3 Lifecycle rules delete uploaded images after 30 days to minimize data retention risk.

7.2 IAM Configuration and Least Privilege

The IAM architecture adheres strictly to the principle of least privilege:

- EC2 Instance Profile: Limited to s3:PutObject, s3:GetObject on the specific result bucket, and CloudWatch log write permissions.
- Developer IAM Users: Separate IAM users for development team members with MFA enforced.

46

- No Root Account Usage: The AWS root account is locked down; all operations use named IAM users or roles.
- Service Control Policies (SCPs): Prevent resource creation outside the ap-south-1 region.

7.3 Scalability

The architecture is designed to scale horizontally:

- Auto Scaling Group: An ASG is configured with a minimum of 1 and maximum of 5 EC2 instances. Scaling is triggered when average CPU utilization exceeds 70%.
- Elastic Load Balancer: An Application Load Balancer distributes incoming requests across healthy instances.
- Stateless Application Design: The Flask application is stateless; session data and model states are not stored per-instance, enabling seamless horizontal scaling.
- S3 Scalability: Amazon S3 handles virtually unlimited concurrent requests, eliminating storage as a bottleneck.

7.4 Reliability and Fault Tolerance

- Multi-AZ Deployment: The Auto Scaling Group spans two Availability Zones (apsouth-1a and ap-south-1b) for high availability.
- EBS Snapshots: Daily automated snapshots of the EC2 EBS volume via AWS Data Lifecycle Manager.
- S3 Durability: Amazon S3 provides 99.999999999% (11 nines) durability through automatic redundancy.
- Health Checks: The ALB performs HTTP health checks on the /health endpoint every 30 seconds. Unhealthy instances are replaced.
- Graceful Error Handling: The Flask application returns structured JSON error responses for all failure cases.

7.5 Cloud Cost Optimization

Table 7.1: Monthly AWS Cost Estimate (Single Instance)

Service	Configuration	Estimated Cost
EC2 (t3.medium)	730 hrs/month	$30.37
EBS (gp3, 50 GB)	50 GB	$4.00
S3 Storage	10 GB	$0.23
S3 Requests	100K PUT/GET	$0.05
Data Transfer	10 GB out	$0.90
CloudFront	50 GB transfer	$4.25
CloudWatch	Basic metrics	$0.00 (free tier)
Total		$39.80/month

Cost optimization strategies include:

- Using Spot Instances for non-production/training workloads (up to 70% savings)
- S3 Intelligent-Tiering to automatically move infrequently accessed objects to cheaper tiers
- Reserved Instances (1-year term) for the production EC2 instance, saving approximately 35%
- CloudWatch Billing Alerts set at 80% of monthly budget

7.6 Monitoring and Logging

- System Metrics: CPU utilization, memory usage, disk I/O, and network throughput are monitored via CloudWatch Agent installed on the EC2 instance.
- Application Metrics: Custom CloudWatch metrics track inference latency, requests per minute, error rate, and S3 upload success rate.
- Log Aggregation: Nginx access/error logs and Gunicorn application logs are shipped to CloudWatch Logs via the CloudWatch Agent.
- Alarms: SNS-based alarms notify the administrator via email when CPU > 80% for 5 minutes or error rate > 5%.

CHAPTER 8

Future Enhancements

8.1 Real-Time Video Deepfake Detection

The current system operates on static images. Extending it to video involves frame-by-frame analysis combined with temporal consistency checking. A future enhancement will integrate 3D-CNNs or Transformer-based architectures (e.g., *ViT-B*) that process spatiotemporal volumes to capture motion-based artifacts. Video ingestion via AWS MediaConvert will extract frames at configurable intervals, and AWS SQS will manage the processing queue for asynchronous inference.

8.2 Blockchain-Based Evidence Integrity

Integrating blockchain technology (e.g., Ethereum or Hyperledger Fabric) would provide tamperproof audit trails for analysis results. When an image is classified as fake, a hash of the image and the detection result can be immutably recorded on-chain, providing cryptographically verifiable evidence admissible in legal proceedings. AWS Managed Blockchain simplifies the deployment of such a distributed ledger.

8.3 Mobile Application

A cross-platform mobile application (React Native or Flutter) would make the system accessible from smartphones. On-device inference using TensorFlow Lite and model quantization (INT8) can enable real-time detection without server round-trips for basic use cases, reducing latency to under 500 milliseconds on modern mobile hardware.

8.4 Multi-Cloud Architecture

To eliminate dependence on a single cloud provider, a future architecture may distribute the workload across AWS, Google Cloud Platform (GCP), and Microsoft Azure. A multi-cloud load balancer (e.g., Cloudflare) routes requests to the nearest available provider. Terraform enables infrastructure-as-code provisioning that is portable across providers.

8.5 Live Streaming Detection

Integration with video streaming platforms (e.g., YouTube Live, Twitch, Facebook Live) via their respective APIs would enable real-time deepfake flagging during live broadcasts. This requires sub-100ms inference latency, achievable through model distillation and deployment on GPU-accelerated Lambda functions or AWS Inferentia chips.

8.6 Explainable AI (XAI) Enhancements

Beyond Grad-CAM, future work will incorporate:

- LIME (Local Interpretable Model-agnostic Explanations): Highlighting superpixel regions most influential in the decision.
- SHAP (SHapley Additive exPlanations): Providing pixel-level attributions with rigorous game-theoretic foundations.
- Counterfactual Explanations: Generating minimally modified versions of the input that would change the classification decision.

8.7 Edge Computing Integration

Deploying a lightweight model variant on AWS IoT Greengrass or NVIDIA Jetson edge devices would enable deepfake detection within content distribution networks, security cameras, or border control systems, reducing bandwidth consumption and cloud processing costs.

CHAPTER 9

Conclusion

This project has successfully demonstrated the design, development, and cloud deployment of a Deepfake Image Detection System that combines state-of-the-art deep learning with production-grade cloud infrastructure. The system addresses a critical challenge in the modern digital information landscape: the rapid proliferation of AI-generated synthetic images that undermine trust, privacy, and democratic accountability.

Technical Achievements: The proposed EfficientNet-B4 based classifier, trained on a diverse 140,000-image dataset comprising FaceForensics++, Celeb-DF v2, and DFDC images, achieved

a test accuracy of 97.4%, AUC-ROC of 0.9924, and F1-score of 97.4%. The twophase training strategy — warm-up followed by fine-tuning of the last 60 backbone layers — proved highly effective in balancing feature extraction quality with adaptation to the deepfake detection task.

System Design and Deployment: The Flask-based REST API, deployed on AWS EC2 with Nginx and Gunicorn, provides a reliable, scalable inference endpoint with a mean response time of 1.43 seconds. The use of AWS S3 for image storage, IAM for security, CloudFront for content delivery, and CloudWatch for monitoring results in a robust, production-grade architecture. The system handles concurrent requests gracefully through Auto Scaling and Elastic Load Balancing.

Interpretability: The integration of Grad-CAM visualization provides transparent, humaninterpretable explanations of detection decisions, highlighting specific facial regions (e.g., eye boundaries, skin texture, hair synthesis artifacts) that inform each classification. This addresses a key limitation of many black-box detection systems and builds user trust.

Cloud Computing Benefits: Cloud deployment provides significant advantages over on-premises alternatives: elasticity to handle variable workloads, pay-as-you-go economics (approximately $40/month for baseline deployment), geographic distribution for low latency, and enterprisegrade reliability. These properties make the system practically deployable by organizations of all sizes.

Broader Impact: As deepfake technology continues to evolve, the methods and architecture presented in this project provide a reusable, extensible foundation for combating synthetic media manipulation. The open design — incorporating publicly available datasets, open-source frameworks, and standard cloud services — ensures that the system can be reproduced, extended, and improved by the research community.

In conclusion, this project affirms that the combination of advanced deep learning and modern cloud computing can yield practical, deployable solutions to emerging challenges in digital media authenticity. Future work will extend the system to video analysis, mobile deployment, and blockchain-based evidence management, further strengthening its applicability to real-world digital forensics.

Bibliography

[1] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial nets,” in *Advances in Neural Information Processing Systems (NIPS)*, vol. 27, pp. 2672–2680, 2014.

[2] D. Afchar, V. Nozick, J. Yamagishi, and I. Echizen, “MesoNet: A compact facial video forgery detection network,” in *Proc. IEEE International Workshop on Information Forensics and Security (WIFS)*, Hong Kong, 2018, pp. 1–7.

[3] A. Rössler, D. Cozzolino, L. Verdoliva, C. Riess, J. Thies, and M. Nießner, “FaceForensics++: Learning to detect manipulated facial images,” in *Proc. IEEE/CVF International Conference on Computer Vision (ICCV)*, Seoul, Korea, 2019, pp. 1–11.

[4] Y. Li, X. Yang, P. Sun, H. Qi, and S. Lyu, “Celeb-DF: A large-scale challenging dataset for deepfake forensics,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Seattle, WA, 2020, pp. 3207–3216.

[5] L. Li and S. Lyu, “Face X-ray for more general face forgery detection,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Seattle, WA, 2020, pp. 5001–5010.

[6] Y. Li, M. Chang, and S. Lyu, “Exposing DeepFake videos by detecting face warping artifacts,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, Long Beach, CA, 2019, pp. 46–52.

[7] H. Dang, F. Liu, J. Stehouwer, X. Liu, and A. K. Jain, “On the detection of digital face manipulation,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Seattle, WA, 2020, pp. 5781–5790.

[8] I. Masi, A. Killekar, R. M. Mascarenhas, S. P. Gurudatt, and W. AbdAlmageed, “Twobranch recurrent network for isolating deepfakes in videos,” in *Proc. European Conference on Computer Vision (ECCV)*, Glasgow, UK, 2020, pp. 667–684.

[9] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *Proc. International Conference on Machine Learning (ICML)*, Long Beach, CA, 2019, pp. 6105–6114.

[10] R. Durall, M. Keuper, F.-J. Pfreundt, and J. Keuper, “Unmasking deepfakes with simple features,” *arXiv preprint arXiv:1911.00686*, 2019.

[11] R. Tolosana, R. Vera-Rodriguez, J. Fierrez, A. Morales, and J. Ortega-Garcia, “Deepfakes and beyond: A survey of face manipulation and fake detection,” *Information Fusion*, vol. 64, pp. 131–148, 2020.

[12] F. Matern, C. Riess, and M. Stamminger, “Exploiting visual artifacts to expose deepfakes and face manipulations,” in *Proc. IEEE Winter Applications of Computer Vision Workshops (WACVW)*, Waikoloa, HI, 2019, pp. 83–92.

[13] T. T. Nguyen, Q. V. H. Nguyen, D. T. Nguyen, D. Nguyen, T. Huynh-The, S. Nahavandi, T. T. Nguyen, Q. L. Pham, and C. M. Nguyen, “Deep learning for deepfakes creation and detection: A survey,” *Computer Vision and Image Understanding*, vol. 223, p. 103525, 2022.

[14] H. Khalid and S. S. Woo, “OC-FakeDect: Classifying deepfakes using one-class variational autoencoder,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, Seattle, WA, 2020, pp. 2794–2803.

[15] K. Raza and M. Singh, “A tour of unsupervised deep learning for medical image analysis,” *Current Medical Imaging*, vol. 17, no. 9, pp. 1059–1077, 2021.

[16] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “GradCAM: Visual explanations from deep networks via gradient-based localization,” in *Proc. IEEE International Conference on Computer Vision (ICCV)*, Venice, Italy, 2017, pp. 618–626.

[17] F. Chollet, “Xception: Deep learning with depthwise separable convolutions,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Honolulu, HI, 2017, pp. 1251–1258.

[18] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, 2016, pp. 770–778.

[19] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” in *Proc. International Conference on Learning Representations (ICLR)*, San Diego, CA, 2015, pp. 1–14.

[20] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

[21] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems (NIPS)*, vol. 25, pp. 1097–1105, 2012.

[22] J. Thies, M. Zollhöfer, M. Stamminger, C. Theobalt, and M. Nießner, “Face2Face: Realtime face capture and reenactment of RGB videos,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, 2016, pp. 2387–2395.

[23] Y. Choi, M.-J. Choi, M. Kim, J.-W. Ha, S. Kim, and J. Choo, “StarGAN: Unified generative adversarial networks for multi-domain image-to-image translation,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Salt Lake City, UT, 2018, pp. 8789–8797.

[24] B. Dolhansky, J. Bitton, B. Pflaum, J. Lu, R. Howes, M. Wang, and C. C. Ferrer, “The DeepFake detection challenge (DFDC) dataset,” *arXiv preprint arXiv:2006.07397*, 2020.

[25] Amazon Web Services, “Amazon EC2 instance types,” AWS Documentation, 2024. [Online]. Available: https://aws.amazon.com/ec2/instance-types/

[26] Amazon Web Services, “Amazon S3 – Cloud Object Storage,” AWS Documentation, 2024. [Online]. Available: https://aws.amazon.com/s3/

[27] Amazon Web Services, “AWS Identity and Access Management (IAM),” AWS Documentation, 2024. [Online]. Available: https://aws.amazon.com/iam/

[28] M. Abadi *et al.*, “TensorFlow: Large-scale machine learning on heterogeneous systems,” *arXiv preprint arXiv:1603.04467*, 2016. [Online]. Available: https://www.tensorflow.org/

[29] Pallets Projects, “Flask – A lightweight WSGI web application framework,” 2024. [Online]. Available: https://flask.palletsprojects.com/

CHAPTER A

Installation and Setup Guide

```
# Step 1: Clone repository
git clone https://github.com/your-org/deepfake-detector.git cd deepfake-detector

# Step 2: Create virtual environment python3.10 -m venv venv
source venv/bin/activate                                    # Windows: venv\Scripts\activate

# Step 3: Install dependencies pip install --
upgrade pip pip install -r requirements.txt

# Step 4: Download pre-trained model weights
# (Obtain from project release page or train from scratch) mkdir -p model/saved_model
# Copy saved_model directory here

# Step 5: Set environment variables export
MODEL_PATH=./model/saved_model export
S3_BUCKET=deepfake-detection-results export
AWS_DEFAULT_REGION=ap-south-1

# Step 6: Run Flask development server cd app python
app.py
# Server starts at http://localhost:5000
```

A.1 Local Development Setup

Listing A.1: Local Environment Setup

```
tensorflow==2.13.0 keras==2.13.0
flask==2.3.3 flask-cors==4.0.0
gunicorn==21.2.0
```

A.2 Requirements File

```
boto3==1.28.0
opencv-python-headless==4.8.0.76
mtcnn==0.1.1 numpy==1.24.3 pandas==2.0.3
matplotlib==3.7.2 scikit-learn==1.3.0
Pillow==10.0.0 tqdm==4.66.1 h5py==3.9.0
```

Listing A.2: requirements.txt

A.3 Docker Deployment

```
FROM python:3.10-slim

WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y \ libglib2.0-0 libsm6 libxext6 libxrender-
        dev \ libgomp1 && \
        rm -rf /var/lib/apt/lists/*

# Copy requirements and install Python packages COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code and model
COPY app/ ./app/
COPY model/saved_model/ ./model/saved_model/

ENV MODEL_PATH=/app/model/saved_model
ENV PYTHONUNBUFFERED=1

EXPOSE 8000

CMD ["gunicorn", "--bind", "0.0.0.0:8000", "--workers", "2", "--timeout", "120",
          "app.app:app"]
```

Listing A.3: Dockerfile

```
# Build Docker image docker build -t deepfake-detector:latest .
```



```
# Run container locally docker run -d \
    -p 8000:8000 \
    -e S3_BUCKET=deepfake-detection-results \
    -e AWS_DEFAULT_REGION=ap-south-1 \
    --name deepfake-app \ deepfake-detector:latest

# Check container logs docker logs
deepfake-app

# Stop and remove container
docker stop deepfake-app && docker rm deepfake-app
```

Listing A.4: Docker Build and Run Commands

CHAPTER B

Sample API Reference

B.1 Analyze Image Endpoint

URL: POST /analyze

Content-Type: multipart/form-data Field:

image (file, required)

```
{
    "record_id": "3f2a7c8e-1b4d-4e9a-a2c1-8f5e3d2b1a9c",
    "label": "Fake",
    "confidence": 96.83,
    "original_url": "https://deepfake-detection-results.s3.amazonaws.com /...",
    "heatmap_url": "https://deepfake-detection-results.s3.amazonaws.com
      /..."
}
```

Success Response (200 OK):

Error Responses:

```
# 400 Bad Request (no file)
{"error": "No image file provided."}

# 422 Unprocessable Entity (no face)
{"error": "No face detected in the image."}

# 413 Payload Too Large (> 10 MB)
{"error": "File size exceeds the 10 MB limit."}
```

B.2 Health Check Endpoint

URL: GET /health

```
{"status": "healthy"}
```

Response (200 OK):

CHAPTER C

AWS S3 Bucket Policy

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "DenyPublicAccess",
            "Effect": "Deny",
            "Principal": "*",
            "Action": "s3:GetObject",
            "Resource": "arn:aws:s3:::deepfake-detection-results/*",
            "Condition": {
                "Bool": {
                    "aws:SecureTransport": "false" }
            }
        }
    ]
}
```

Listing C.1: S3 Bucket Policy